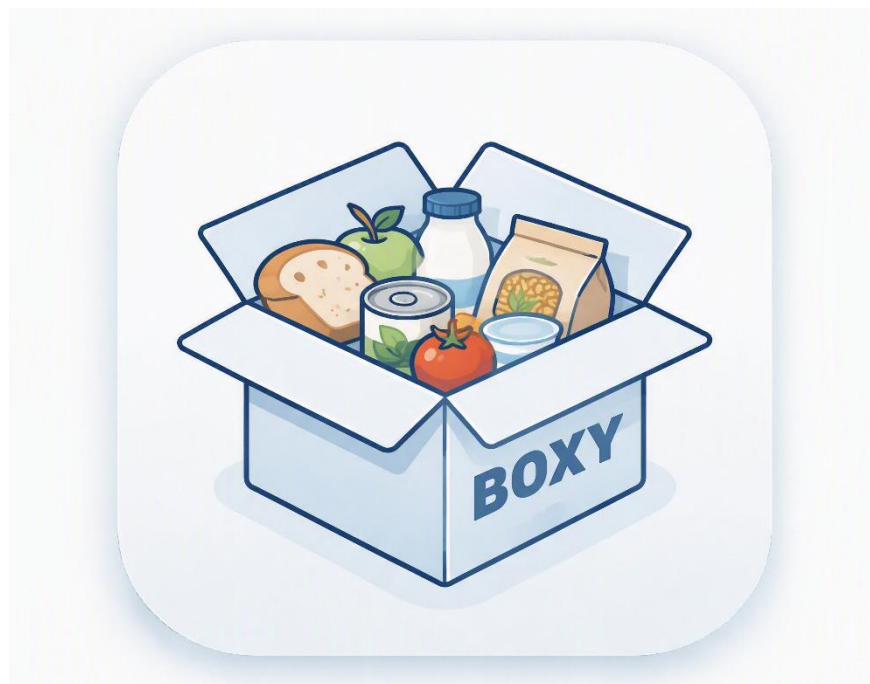


Documentazione del Progetto: Programmazione Object Oriented

BOXY

Sistema software per la gestione e la compravendita di eccedenze alimentari



Roberta Pagliuca (DE100058)

Noemi Savelli (DE100321)

Indice

II. Diagramma delle Classi e Presentazione del Dominio	5
II.A Class Diagram.....	5
II.B Classi del Dominio.....	5
II.C Associazioni e Cardinalità	6
II.D Regole di Business	6
III Lista delle Funzioni ad Alto Livello.....	7
III.A Modulo 1: Gestione Utenti ed Autenticazione.....	7
• registraUtente:.....	7
• validaLogin:	7
• modificaCredenzialiUtente:.....	7
• eliminaAccount:	7
III.B Modulo 2: Gestione Punti Vendita e Catalogo Box.....	7
• visualizzaPuntiVendita:	7
• visualizzaBoxDisponibili:	7
III.C Modulo 3: Gestione Prenotazioni e Ordini	8
• effettuaPrenotazione:	8
• visualizzaPrenotazioniUtente:	8
• annullaPrenotazione:	8
IV Architettura Software e Struttura dei Package.....	9
IV.A BCE + DAO Pattern.	9
IV.B Singleton Pattern per la Persistenza	9
IV.C Incapsulamento e Disaccoppiamento	10
V Gestione delle Eccezioni.....	11
V.A Gestione su più Livelli	11
V.B Eccezioni Personalizzate.....	11
VI Sequence Diagram delle Operazioni Critiche	12
VI.A Operazione 1: effettuaPrenotazione	12
VI.B Operazione 2: annullaPrenotazione	13
VII Repository GitHub	15

I. Introduzione e Requisiti del Sistema

I.A Presentazione del dominio scelto

Il progetto BOXY si colloca nel contesto della sostenibilità e della lotta allo spreco alimentare, traendo ispirazione dal modello di business delle food-bag a sorpresa (come Too Good To Go). Il sistema software si pone come tramite tra le attività commerciali aderenti (ristoranti, supermercati, forni e bar) e gli acquirenti finali. L'applicazione consente ai punti vendita di catalogare e mettere a disposizione lotti di cibo invenduto a prezzo scontato, permettendo agli utenti registrati di localizzare i negozi, consultare le offerte attive, prenotare le box e ritirarle nella fascia oraria prestabilita.

I.B Scopo del progetto

L'obiettivo è digitalizzare l'intero flusso di acquisto e ritiro delle eccedenze alimentari. Nello specifico, il sistema permette di:

- Registrare un nuovo utente e gestirne l'autenticazione tramite credenziali e personalizzazione dell'avatar di profilo.
- Esplorare i punti vendita partner filtrandoli per tipologia (Ristoranti, Supermercati).
- Consultare la scheda descrittiva di ciascuna box (fascia oraria di ritiro, indirizzo del punto vendita, prezzo originario, prezzo scontato e quantità disponibile).
- Selezionare e prenotare una o più box inserendo il quantitativo desiderato.
- Gestire la cronologia delle prenotazioni nella sezione personale, con facoltà di visionare lo stato dell'ordine o effettuarne l'annullamento.

I.C Requisiti funzionali (Casi d'Uso)

Dal punto di vista operativo, il software espone i seguenti casi d'uso principali:

1. **Autenticazione e Profilo:** registrazione, login con verifica credenziali, modifica dati anagrafici e selezione dell'avatar.
2. **Consultazione Catalogo:** visualizzazione dei punti vendita registrati e navigazione tra le relative box disponibili in magazzino.
3. **Gestione Ordine:** selezione del numero di box da acquistare, controllo della disponibilità e conferma della prenotazione.
4. **Monitoraggio e Storno:** visualizzazione del riepilogo "Le mie box" e annullamento della prenotazione attiva con contestuale ripristino delle quantità a catalogo.

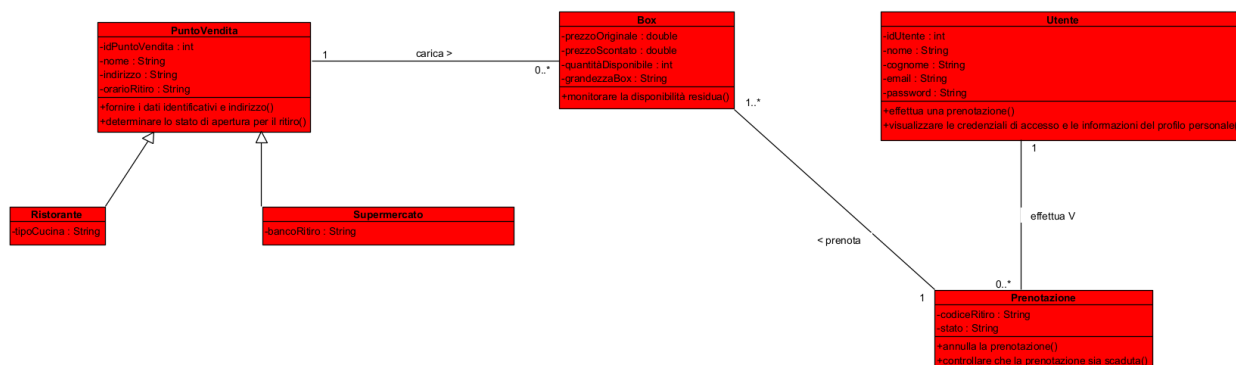
I.D Requisiti di Progettazione Object-Oriented

La progettazione dell'applicativo rispetta i principi cardine dell'Object-Oriented Programming (OOP):

- **Incapsulamento:** gli attributi di tutte le classi di dominio sono stati dichiarati private, garantendo l'accesso e la modifica dello stato interno esclusivamente tramite metodi pubblici di getter e setter con opportuni controlli di validità.
- **Ereditarietà:** impiego di gerarchie di classi sia nell'architettura della GUI (estendendo i componenti nativi di Java Swing come JFrame e JPanel) sia nella modellazione del dominio per condividere proprietà e comportamenti tra entità correlate.
- **Polimorfismo:** utilizzo di interfacce per separare la logica applicativa dall'accesso ai dati (pattern DAO), consentendo al programma di invocare metodi astratti la cui esecuzione specifica viene risolta dinamicamente a runtime.
- **Affidabilità:** il programma è in grado di gestire qualsiasi errore legato al database o al funzionamento generale, così da evitare di perdere i dati dell'utente.
- **Architettura software:** il codice è stato organizzato seguendo il pattern architetturale BCE insieme alla gestione DAO.
- **Separazione dei ruoli:** il sistema è stato diviso in package separati per tenere in ordine la parte grafica (GUI), la parte di controllo della logica (Controller), i dati (Model) e la connessione al database (DAO).

II. Diagramma delle Classi e Presentazione del Dominio

II.A Class Diagram



Lo schema concettuale rappresenta il dominio applicativo in formato astratto.

Figura 1: Diagramma delle classi UML del sistema Boxy

II.B Classi del Dominio

All'interno del package model sono state inserite le classi ricavate dallo schema concettuale, che costituiscono le entit  principali e fondamentali del sistema:

Utente: Rappresenta il cliente registrato nel sistema.

- **Attributi:** idUtente, nome, cognome, e-mail, password.
- **Metodi:** effettuaUnaPrenotazione(), visualizzare le credenziali di accesso e le informazioni del profilo personale.

PuntoVendita: Rappresenta l'attivit  commerciale generica (ristorante o supermercato) che offre le box.

- **Attributi:** idPuntoVendita, nome, indirizzo, orarioRitiro.
- **Metodi:** fornire i dati identificativi e indirizzo, monitorare lo stato di apertura per il ritiro.

Ristorante (Sottoclasse di PuntoVendita): Tipologia specifica di punto vendita di ristorazione.

- **Attributi:** tipoCucina.

Supermercato (Sottoclasse di PuntoVendita): Tipologia specifica di punto vendita.

- **Attributi:** bancoRitiro.

Box: Rappresenta la box anti-spreco alimentare messa a disposizione dai punti vendita.

- **Attributi:** prezzoOriginale, prezzoScontato, quantitaDisponibile, grandezzaBox.
- **Metodi:** monitorare la disponibilit  residua.

Prenotazione: Rappresenta l'ordine o la prenotazione effettuata dall'utente per una o pi  box.

- **Attributi:** codiceRitiro, stato.
- **Metodi:** annulla la prenotazione, controllare che la prenotazione sia scaduta.

II.C Associazioni e Cardinalità

- **Generalizzazione PuntoVendita:** PuntoVendita si generalizza in Ristorante e Supermercato. Si tratta di una generalizzazione totale (tutti i punti vendita appartengono a una delle categorie) ed esclusiva (un punto vendita non può essere contemporaneamente un ristorante e un supermercato).
- **PuntoVendita-Box (1 a 0..*):** un punto vendita può caricare zero o più box e ogni box appartiene a un solo punto vendita.
- **Utente-Prenotazione (1 a 0..*):** un utente può effettuare zero o più prenotazioni e ogni prenotazione è associata a un singolo utente.
- **Box-Prenotazione (1..* a 1):** una prenotazione riguarda una o più box e ogni box può essere inserita in una prenotazione.

II.D Regole di Business

- Un Utente potrà prenotare/acquistare una Box solo se la quantità disponibile è maggiore di zero.
- Gli attributi nome, cognome, e-mail e password di Utente sono stringhe con lunghezza non superiore ai limiti previsti dai form di input.
- I prezzi prezzoOriginale e prezzoScontato di Box devono essere valori numerici maggiori o uguali a zero, con il prezzo scontato inferiore o uguale a quello originale.
- Un Utente non può effettuare prenotazioni multiple identiche non gestite o sovrapposte per lo stesso punto vendita senza disponibilità.
- Gli orari di ritiro indicati nei punti vendita devono rispettare la coerenza temporale della giornata corrente.

III Lista delle Funzioni ad Alto Livello

Le funzioni ad alto livello sono raggruppate in tre moduli logici:

III.A Modulo 1: Gestione Utenti ed Autenticazione

- **registraUtente:**
 - *Descrizione:* Registra un nuovo account utente salvando i dati anagrafici.
 - *Input:* Oggetto Utente contenente nome, cognome, e-mail e password.
 - *Output:* void (inserimento eseguito).
- **validaLogin:**
 - *Descrizione:* Verifica la corrispondenza delle credenziali inserite dall'utente per consentire l'accesso.
 - *Input:* e-mail (String), password (String).
 - *Output:* boolean (esito dell'autenticazione)
- **modificaCredenzialiUtente:**
 - *Descrizione:* Permette all'utente di aggiornare le proprie informazioni personali, il nome, il cognome, l'e-mail, la password o l'avatar.
 - *Input:* Oggetto Utente, nuovoNome (String), nuovoCognome (String), nuovaEmail (String), nuovaPassword (String), nuovoAvatar (String).
 - *Output:* void (aggiornamento eseguito).
- **eliminaAccount:**
 - *Descrizione:* Elimina definitivamente l'account dell'utente dal sistema.
 - *Input:* Oggetto Utente.
 - *Output:* void (eliminazione eseguita).

III.B Modulo 2: Gestione Punti Vendita e Catalogo Box

- **visualizzaPuntiVendita:**
 - *Descrizione:* Recupera l'elenco dei punti vendita suddivisi per tipologia (Ristoranti o Supermercati).
 - *Input:* tipologia (String).
 - *Output:* ArrayList<PuntoVendita>.
- **visualizzaBoxDisponibili:**
 - *Descrizione:* Mostra l'elenco delle box offerte da un determinato punto vendita con tutti i relativi dettagli.
 - *Input:* idPuntoVendita (int).
 - *Output:* ArrayList <Box>.

III.C Modulo 3: Gestione Prenotazioni e Ordini

- **effettuaPrenotazione:**
 - *Descrizione:* Registra l'acquisto di una o più box da parte dell'utente aggiornando la quantità disponibile.
 - *Input:* Oggetto Utente, Oggetto Box, Quantita (int).
 - *Output:* void (prenotazione registrata).
- **visualizzaPrenotazioniUtente:**
 - *Descrizione:* Restituisce l'elenco di tutte le box prenotate e attive associate all'utente ("Le mie box!!").
 - *Input:* Oggetto Utente.
 - *Output:* ArrayList <Prenotazione>.
- **annullaPrenotazione:**
 - *Descrizione:* Cancella una prenotazione attiva effettuata dall'utente e ripristina la disponibilità della box.
 - *Input:* Oggetto Prenotazione.
 - *Output:* void (annullamento eseguito).

IV Architettura Software e Struttura dei Package

IV.A BCE + DAO Pattern.

Per mantenere la modularità del codice e separare le responsabilità, il progetto Boxy è diviso in livelli logici distinti (BCE), sfruttando le interfacce DAO per i collegamenti al database:

- **Package gui (Boundary):** Gestisce la parte grafica del sistema e ciò che l'utente vede a schermo (es. Home, Login, Prenotazione, RistoranteForm , Supermercato). I componenti grafici non comunicano mai direttamente con il database, ma passano sempre attraverso il livello di controllo.
- **Package controller (Control):** Contiene la classe Controller, che funge da mediatore all'interno dell'applicazione: applica le logiche di business, riceve le richieste dalla grafica, effettua i controlli e chiama i DAO per salvare o leggere i dati.
- **Package model (Entity):** Contiene le classi di entità effettive dei dati del sistema anti-spreco (es. Box, Utente, Prenotazione).
- **Package dao e implementazionePostgresDAO (DAO):** Per non vincolare il programma a un database specifico, vengono definite le interfacce nel package dao (es. BoxDAO, UtenteDAO, PrenotazioneDAO) che vengono poi implementate concretamente nel package implementazionepostgresDAO (es. BoxPostgresDAO, UtentePostgresDAO, PrenotazionePostgresDAO) usando codice SQL e connessioni JDBC.
- **Package database:** Contiene la classe ConnessioneDatabase, che gestisce il collegamento fisico e centralizzato al database PostgreSQL.

IV.B Singleton Pattern per la Persistenza

Per evitare l'apertura di molteplici connessioni al database (che causano rallentamenti del programma e consumo di memoria), è stato utilizzato il pattern **Singleton** nella classe ConnessioneDatabase. Questo pattern fa sì che venga creata una sola istanza di connessione per tutta l'applicazione, e che venga riutilizzata in modo sicuro:

Listing 1: Il Singleton in ConnessioneDatabase.java

```
public class ConnessioneDatabase {

    //istanza statica per il singleton
    private static ConnessioneDatabase instance;

    //attributi per la connessione al database
    private Connection connection = null;
    private final String nome = "postgres";
    private final String password = "password";
    private final String url = "jdbc:postgresql://localhost:5432/Boxy";
    private final String driver = "org.postgresql.Driver";

    //costruttore privato per impedire la creazione diretta di altre istanze
    private ConnessioneDatabase() throws SQLException {
        try {
            Class.forName(driver);
            connection = DriverManager.getConnection(url, nome, password);
        } catch (ClassNotFoundException ex) {
            System.out.println("errore caricamento driver: " + ex.getMessage());
        }
    }

    //metodo per ottenere l'unica istanza attiva della connessione
    public static ConnessioneDatabase getInstance() throws SQLException {
        if (instance == null) {
            instance = new ConnessioneDatabase();
        } else if (instance.connection == null || instance.connection.isClosed()) {
            instance = new ConnessioneDatabase();
        }
        return instance;
    }

    //metodo get per ottenere la connessione rispettando l'incapsulamento
    public Connection getConnection() {
        return connection;
    }
}
```

IV. Incapsulamento e Disaccoppiamento

Il sistema applica rigorosamente l'incapsulamento impedendo la manipolazione diretta del database o delle liste in memoria. I DAO incapsulano le query SQL parametriche tramite PreparedStatement (come implementato nelle classi di persistenza), impedendo vulnerabilità di tipo SQL Injection e garantendo la corretta persistenza delle box, degli utenti e delle prenotazioni. Inoltre, il Controller disaccoppia nettamente l'interfaccia grafica dai dati, gestendo la logica di business ed evitando dipendenze dirette tra la GUI e il database.

V Gestione delle Eccezioni

V.A Gestione su più Livelli

Per fare in modo che l'applicazione risulti sicura e robusta, le eccezioni vengono gestite dividendo i ruoli tra i vari package. Quando si verifica un errore durante le operazioni sul database (ad esempio un problema di connessione o un inserimento non valido), il codice JDBC solleva una `SQLException`.

- I. Il package di implementazione del database propaga l'eccezione marcando il metodo con `throws SQLException`.
- II. Il Controller intercetta l'errore e gestisce la logica applicativa associata.
- III. La classe della grafica (gui) cattura l'eccezione e mostra il relativo messaggio di errore all'utente a schermo.

V.B Eccezioni Personalizzate

Per gestire i controlli e le situazioni specifiche della nostra applicazione anti-spreco, abbiamo creato 5 classi di eccezione personalizzate all'interno del package gui:

- I. `ExceptionCognome (RuntimeException)`: Sollevata quando il campo del cognome non rispetta i requisiti previsti in fase di registrazione o modifica.
- II. `ExceptionEmail (RuntimeException)`: Sollevata se l'indirizzo e-mail inserito non è valido o risulta errato.
- III. `ExceptionEmailUguale (RuntimeException)`: Sollevata nel caso in cui si tenti di registrare un account con un'e-mail già esistente nel sistema.
- IV. `ExceptionNome (RuntimeException)`: Sollevata quando il campo del nome non risulta valido o conforme ai controlli.
- V. `ExceptionPassword (RuntimeException)`: Sollevata in caso di errori di inserimento o validazione della password.

VI Sequence Diagram delle Operazioni Critiche

In questa sezione vengono descritte le operazioni critiche del sistema, illustrando i relativi sequence diagram multilivello (Controller/GUI, DAO e Modello) e lo pseudocodice logico dell'operazione.

VI.A Operazione 1: effettuaPrenotazione

Questa operazione finalizza la prenotazione delle box anti-spreco selezionate dall'utente loggato, verificando la disponibilità e aggiornando le scorte nel database.

- **Signature:** public boolean inserisciPrenotazione(Prenotazione prenotazione, String emailUtente, int idBox) throws SQLException
- **Logica di Business:** Il controller ricevi la richiesta di prenotazione dall'interfaccia grafica, valida i dati e si interfaccia con il livello DAO per inserire la prenotazione associandola all'utente e decrementando contestualmente la quantità disponibile della box.
- **Operazioni DB:** Inserimento di record nella tabella prenotazione ed esecuzione di un aggiornamento (UPDATE) sulla tabella box tramite PrenotazionePostgresDAO e BoxPostgresDAO.

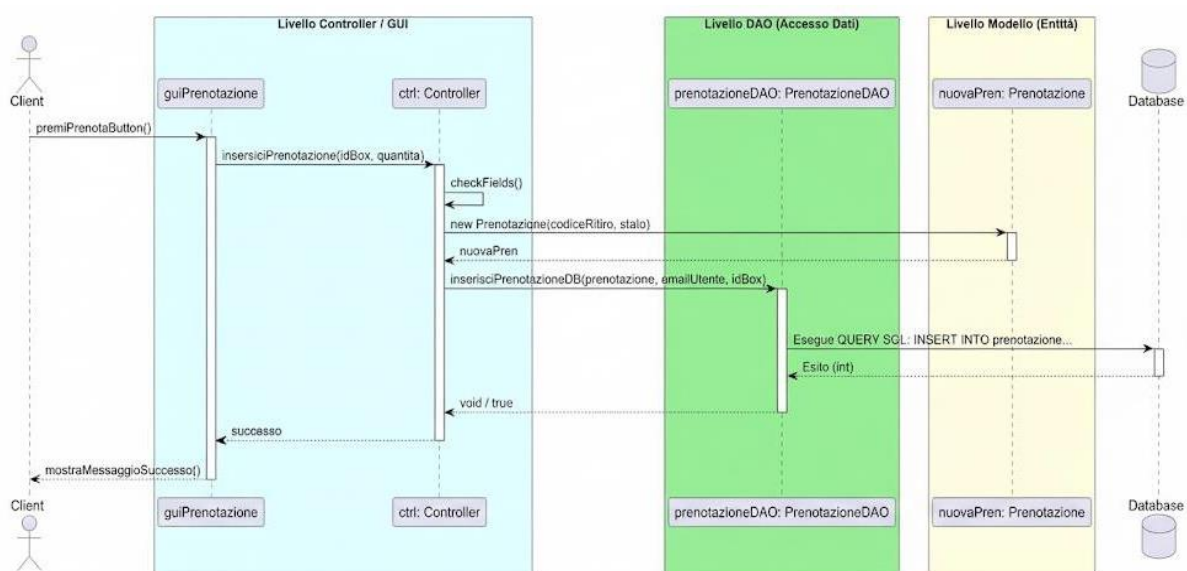


Figura 2: Diagramma di sequenza dell'operazione effettuaPrenotazione

Listing 2: Pseudocodice dell'operazione acquistaBoxDB

```
// Nel Controller: acquistaBoxDB(idBox)
IF utenteAttuale == NULL THEN
    RETURN NULL
ENDIF

TRY
    // Generazione codice univoco e creazione istanza prenotazione
    codiceUnivoco = "BOX-" + tempoCorrenteInMillisecondi()
    nuovaPrenotazione = NEW Prenotazione(codiceUnivoco)
    nuovaPrenotazione.setStato("ATTIVA")

    // Inserimento persistente tramite PrenotazioneDAO
    inserito = prenotazioneDAO.inserisciPrenotazione(nuovaPrenotazione,
    utenteAttuale.getEmail(), idBox)

    IF inserito == TRUE THEN
        RETURN codiceUnivoco
    ENDIF
    RETURN NULL

CATCH SQLException e
    MOSTRA "errore acquisto box: " + e.getMessage()
    RETURN NULL

ENDTRY
```

VI.B Operazione 2: annullaPrenotazione

Questa operazione consente all'utente di cancellare una prenotazione attiva all'interno del sistema Boxy, aggiornando lo stato nel database e ripristinando la disponibilità della box nel punto vendita.

- **Signature:** public boolean annullaPrenotazione(int idBox, String emailUtente) throws SQLException
- **Logica di Business:** Il controller riceve la richiesta di annullamento dall'interfaccia grafica, verifica che i parametri siano corretti e si interfaccia con il livello DAO per individuare la prenotazione nello stato "ATTIVA" ed eseguirne la disdetta logica.
- **Operazioni DB:** Chiamata a PrenotazionePostgresDAO.annullaPrenotazione(idBox, emailUtente), che esegue una query di aggiornamento (UPDATE) dello stato della prenotazione sul database.

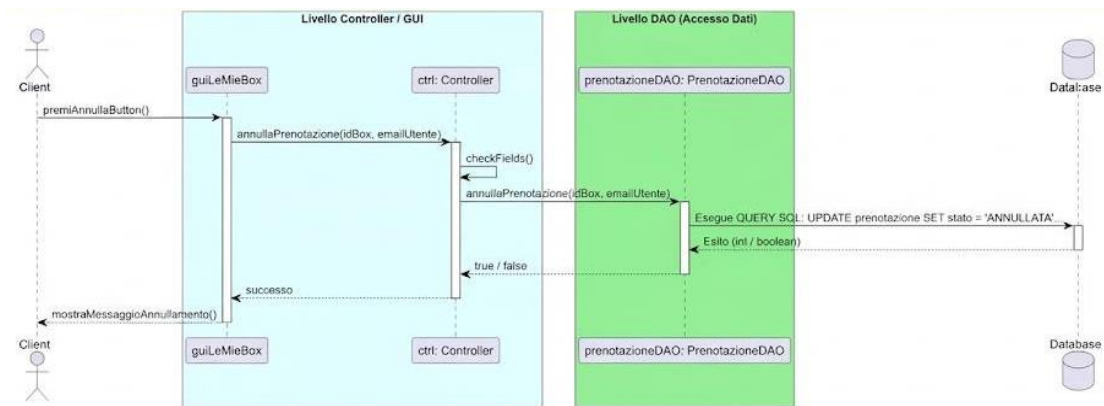


Figura 3: Diagramma di sequenza dell'operazione annullaPrenotazione

Listing 3: Pseudocodice dell'operazione annullaPrenotazioneDB

```

// Nel Controller: annullaPrenotazioneDB(idBox)
IF utenteAttuale == NULL THEN
    RETURN FALSE
ENDIF

TRY
    // Chiamata al DAO per aggiornare lo stato nel database relazionale
    esito = prenotazioneDAO.annullaPrenotazione(idBox,
    utenteAttuale.getEmail())
    RETURN esito

CATCH SQLException e
    MOSTRA "errore annullamento prenotazione: " + e.getMessage()
    RETURN FALSE
ENDTRY
  
```

VII Repository GitHub

- **Link alla repository del progetto:**

<https://github.com/pagliuca26/Progetto-POO-BD-2025-2026>